



# 1. Python 語法入門

Python 是一種語法簡潔、容易閱讀的程式語言。這個章節介紹在後端開發中最常用到的語法，讓你看到程式碼時不會陌生。

## 1.1 變數與資料型別

Python 不需要宣告型別，直接賦值就好。常用的型別有以下幾種：

```
# 字串 (str)
name = '王小明'

# 整數 (int)
age = 25

# 浮點數 (float)
weight = 65.5

# 布林值 (bool)
is_eligible = True

# None：代表「沒有值」，類似 SQL 的 NULL
last_donation = None
```

Python 用縮排(空格)來表示程式碼的層次，不用大括號 {}。這點和大部分語言不同，要特別注意。

## 1.2 字典(dict)與串列(list)

這兩個結構在後端開發裡非常重要。從資料庫查出來的資料通常就是這兩種格式。

字典(dict) — 對應到一筆資料庫記錄

字典用來儲存「鍵值對」，結構和 JSON 很像：

```
# 一筆使用者資料
user = {
    'donor_id': 1,
    'name': '王小明',
    'blood_type': 'A',
    'hold_points': 150
}

# 用鍵 (key) 取值
print(user['name']) # 王小明
print(user['hold_points']) # 150

# 修改值
user['hold_points'] = 200

# 新增鍵值
user['email'] = 'wang@example.com'
```

## 串列(list)—對應到多筆資料庫記錄

串列用來儲存多個項目，資料庫查詢多筆結果時會回傳這種格式：

```
# 多筆使用者
users = [
    {'donor_id': 1, 'name': '王小明'},
    {'donor_id': 2, 'name': '李大華'},
    {'donor_id': 3, 'name': '張美玲'},
]

# 用索引取第一筆 (索引從 0 開始)
print(users[0])          # {'donor_id': 1, 'name': '王小明'}
print(users[0]['name'])  # 王小明

# 走訪所有筆資料
for user in users:
    print(user['name'])   # 依序印出每個人的名字
```

## 1.3 函數(Function)

函數是把一段程式碼包起來，給它一個名字，之後可以重複呼叫。在 FastAPI 裡，每一支 API 就是一個函數。

```
# 定義函數：用 def 關鍵字
def greet(name):          # name 是參數
    message = '你好, ' + name
    return message        # return 是回傳值

# 呼叫函數
result = greet('小明')    # result = '你好, 小明'

# 有預設值的參數 (呼叫時可以不傳)
def get_records(donor_id, limit=20):
    # 如果呼叫時沒有傳 limit, 就用預設值 20
    pass                  # pass 代表「這裡先留空」

get_records(1)            # limit 用預設值 20
get_records(1, 5)         # limit 改成 5
```

## 1.4 條件判斷(if / else)

根據條件決定要執行哪段程式碼：

```
points = 250

if points < 100:
    level = '捐血新鮮人'
elif points < 300:          # elif = else if
    level = '捐血小天使'
elif points < 600:
    level = '捐血達人'
else:
    level = '捐血英雄'

print(level)               # 捐血小天使

# 判斷是否為 None
last_date = None
```

```
if last_date is None:
    print('從未捐過血')
else:
    print('上次捐血日期：', last_date)
```

## 1.5 錯誤處理 (try / except / finally)

當程式碼可能發生錯誤時 (例如資料庫連線失敗)，用 try/except 來捕捉錯誤，避免整個程式崩潰。這是後端開發裡非常重要的習慣。

```
try:
    # 把「可能會出錯」的程式碼放在這裡
    result = 10 / 0    # 這行會產生除以零的錯誤
    print(result)

except ZeroDivisionError:
    # 當發生特定錯誤時執行這裡
    print('不能除以零！')

except Exception as e:
    # 捕捉所有其他類型的錯誤
    # e 包含錯誤的詳細訊息
    print('發生未知錯誤：', str(e))

finally:
    # 無論成功或失敗，這裡一定會執行
    # 通常用來關閉資源，例如資料庫連線
    print('程式結束')
```

### 為什麼一定要有 finally？

在資料庫操作中，finally 的主要用途是確保連線一定會被關閉。  
如果沒有 finally，程式發生錯誤時可能會忘記關閉連線，  
久了之後連線數量累積過多，資料庫會拒絕新的連線請求。

## 1.6 選填欄位 (Optional)

在 Python 的型別提示中，Optional 代表這個值可能是指定型別，也可能是 None，對應到資料庫裡允許 NULL 的欄位。

```
from typing import Optional

# 這個函數的 phone 參數可以傳字串，也可以不傳 (預設 None)
def create_user(name: str, phone: Optional[str] = None):
    if phone is None:
        print('沒有填電話')
    else:
        print('電話：', phone)
```

## 2. 後端開發常用的 Python 技巧

這個章節介紹在 FastAPI 後端開發裡特別常用的幾個技巧，遇到不熟悉的語法時可以回來查。

### 2.1 f-string: 字串格式化

f-string 是 Python 組合字串最常用的方式，在字串前加 f，然後用 {} 包住變數：

```
name = '王小明'
points = 200

# f-string
message = f'恭喜 {name} 累積了 {points} 點！'
# 結果：'恭喜 王小明 累積了 200 點！'

# 也可以在 {} 裡做計算
message2 = f'你還差 {300 - points} 點才能升級'
# 結果：'你還差 100 點才能升級'
```

### 2.2 字典的常用操作

從資料庫拿到的資料是字典格式，這些操作很常用：

```
user = {'donor_id': 1, 'name': '王小明', 'weight': 65.0}

# 安全取值：如果 key 不存在，不會報錯，回傳預設值
email = user.get('email', '未填寫') # '未填寫'

# 取得所有的 key
keys = user.keys() # dict_keys(['donor_id', 'name', 'weight'])

# 取得所有的 value
values = user.values() # dict_values([1, '王小明', 65.0])

# 同時走訪 key 和 value
for key, value in user.items():
    print(f'{key}: {value}')
# donor_id: 1
# name: 王小明
# weight: 65.0
```

### 2.3 動態組合 SQL 的技巧

更新資料時，使用者不一定會填所有欄位，所以 SQL 的 SET 子句需要動態組合。這個技巧在後面的模組開發裡很重要：

```
# 假設使用者只傳了部分欄位要更新
fields_to_update = {
    'weight': 68.0,
    'phone': '0912345678'
}

# 目標：組合出 'weight = %s, phone = %s' 這樣的字串

# 用 join 把每個 key 組合成 'key = %s' 的格式
set_clause = ', '.join(f'{key} = %s' for key in fields_to_update)
# 結果：'weight = %s, phone = %s'

# 把值取出來，加上 WHERE 條件的 ID
```

```
values = list(fields_to_update.values()) + [donor_id]
# 結果: [68.0, '0912345678', 1]

# 最後組成完整 SQL
sql = f'UPDATE users SET {set_clause} WHERE donor_id = %s'
# 'UPDATE users SET weight = %s, phone = %s WHERE donor_id = %s'
```

### ⚠ 注意

只有 SET 子句用 f-string 組合(因為欄位名稱是我們自己控制的), 值(%s)一定要用參數化查詢傳入, 不能直接塞進字串。  
這個重要觀念在第 4 章會詳細說明。

## 2.4 環境變數與 .env 檔案

資料庫的帳號密碼不能直接寫在程式碼裡, 要存在 .env 檔案, 再用程式讀取。這樣可以避免帳號不小心上傳到 Git:

```
# .env 檔案內容 (這個檔案不上傳到 Git)
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=你的密碼
DB_NAME=blood_donation_db
```

```
# 在 Python 裡讀取 .env
from dotenv import load_dotenv
import os

load_dotenv() # 載入 .env 檔案

host = os.getenv('DB_HOST') # 'localhost'
password = os.getenv('DB_PASSWORD') # 你的密碼
```

## 3. FastAPI 框架介紹

FastAPI 是一套用 Python 撰寫的 Web API 框架。它的工作是「接收來自前端的請求，執行對應的程式碼，再把結果回傳給前端」。

### 3.1 HTTP 方法與 CRUD 的對應

後端 API 的四種基本操作，對應到四種 HTTP 方法：

| HTTP 方法 | 資料庫操作  | 使用情境                |
|---------|--------|---------------------|
| GET     | SELECT | 讀取資料，例如查詢使用者資料、捐血紀錄 |
| POST    | INSERT | 新增資料，例如新增使用者、新增捐血紀錄 |
| PUT     | UPDATE | 更新資料，例如修改體重、更新點數    |
| DELETE  | DELETE | 刪除資料，例如刪除錯誤的捐血紀錄    |

### 3.2 一支 API 的基本結構

FastAPI 用裝飾器 (@ 開頭的語法) 來定義路由，告訴系統「當收到這個 URL 的請求，就執行下面的函數」：

```
from fastapi import APIRouter

router = APIRouter()

# @router.get → 這支 API 接受 GET 請求
# '/items/{item_id}' → URL 路徑, {item_id} 是路徑參數
@router.get('/items/{item_id}')
def get_item(item_id: int):    # FastAPI 自動把路徑參數轉成 int
    # 在這裡寫查詢資料庫的程式碼
    # ...
    return {'item_id': item_id, 'name': '某個物品'}
    # FastAPI 自動把 dict 轉成 JSON 回傳
```

URL 路徑的設計原則：

- 用名詞代表資源，不用動詞：**/users**，不要 **/getUsers**
- 路徑參數放在 {} 裡，代表某個特定資源：**/users/{donor\_id}**
- 查詢參數(非必要的篩選條件)用 ? 帶入：**/sites?category=station**

### 3.3 路徑參數與查詢參數

```
from fastapi import APIRouter, Query
from typing import Optional

router = APIRouter()
```

```

# 路徑參數：放在 URL 裡，一定要傳
# GET /users/1 → donor_id = 1
@router.get('/users/{donor_id}')
def get_user(donor_id: int):
    pass

# 查詢參數：放在 URL 後面的 ?key=value, 可以是選填
# GET /sites → category = None (不篩選)
# GET /sites?category=station → category = 'station'
@router.get('/sites')
def list_sites(category: Optional[str] = None):
    pass

# 同時有兩種參數
# GET /donations/user/1?limit=10
@router.get('/donations/user/{donor_id}')
def get_donations(donor_id: int, limit: int = 20):
    pass

```

### 3.4 Pydantic Schema: 定義資料格式

Pydantic 用來定義「前端要送什麼格式的資料」以及「後端要回傳什麼格式」。FastAPI 會自動驗證，格式不對的話會直接回傳 422 錯誤。

```

from pydantic import BaseModel
from typing import Optional
from datetime import date

# 新增時用的格式 (不含 ID, ID 由資料庫自動產生)
class ItemCreate(BaseModel):
    name: str # 必填
    description: Optional[str] = None # 選填, 沒傳就是 None
    price: float # 必填

# 更新時用的格式 (所有欄位都是選填)
class ItemUpdate(BaseModel):
    name: Optional[str] = None
    price: Optional[float] = None

# API 接收 POST 請求時, 自動解析 JSON body
@router.post('/items')
def create_item(data: ItemCreate): # data 就是前端送來的 JSON
    print(data.name) # 用點 (.) 取屬性
    print(data.price)

# model_dump() 把 Pydantic 物件轉成 dict
# exclude_unset=True 只取有明確傳入的欄位 (更新時很重要)
fields = data.model_dump(exclude_unset=True)
# 如果只傳了 price, fields 就只有 {'price': 新價格}

```

### 3.5 HTTPException: 回傳錯誤



當資料不存在或輸入有問題時，用 HTTPException 回傳對應的 HTTP 狀態碼：

```
from fastapi import HTTPException

@router.get('/users/{donor_id}')
def get_user(donor_id: int):
    # 假設查詢資料庫後沒找到
    user = None

    if not user:
        # status_code=404 表示「找不到」
        # detail 是回傳給前端的錯誤訊息
        raise HTTPException(status_code=404, detail='找不到此使用者')

    return user
```

| 狀態碼                       | 意義與使用時機                             |
|---------------------------|-------------------------------------|
| 200 OK                    | 成功(GET、PUT 的預設值)                    |
| 201 Created               | 新增成功(POST 用, 要明確指定 status_code=201) |
| 204 No Content            | 刪除成功, 不回傳任何內容(DELETE 用)             |
| 400 Bad Request           | 請求有誤, 例如 email 重複、點數不足              |
| 404 Not Found             | 找不到指定資源, 例如查詢不存在的 donor_id          |
| 500 Internal Server Error | 伺服器內部錯誤, 通常是資料庫出問題                  |

## 4. 用 Python 連接 MySQL

這個章節介紹如何用 mysql-connector-python 建立資料庫連線，以及執行 SQL 的正確方式。

### 4.1 建立連線

每次執行 SQL 前要建立連線，用完後一定要關閉：

```
import mysql.connector

# 建立連線
conn = mysql.connector.connect(
    host='localhost',
    port=3306,
    user='root',
    password='你的密碼',
    database='blood_donation_db',
    charset='utf8mb4' # 支援中文
)

# 建立 cursor (游標)，用來執行 SQL
# dictionary=True 讓查詢結果是 dict 格式，而不是 tuple
cursor = conn.cursor(dictionary=True)

# ... 執行 SQL ...

# 用完一定要關閉
cursor.close()
conn.close()
```

#### ? 為什麼要 dictionary=True?

不加: 查詢結果是 tuple, 例如 (1, '王小明', 'A'), 要用 row[0], row[1] 取值, 很難讀。  
 加了: 查詢結果是 dict, 例如 {'donor\_id': 1, 'name': '王小明'}, 直接用欄位名稱取值。  
 後端幾乎所有情境都要加 dictionary=True。

### 4.2 參數化查詢 (防止 SQL Injection)

這是整個指南最重要的觀念。絕對不要把變數直接塞進 SQL 字串，一定要用 %s 佔位符搭配 tuple 傳入：

```
# ❌ 危險寫法 - 不能這樣做！
# 如果 email 是 "' OR 1=1 --", 資料庫會被攻擊
sql = f"SELECT * FROM users WHERE email = '{email}'"
cursor.execute(sql)

# ✅ 正確寫法 - 參數化查詢
sql = 'SELECT * FROM users WHERE email = %s'
cursor.execute(sql, (email,)) # 注意：單一參數也要用 tuple, 要加逗號

# 多個參數
sql = 'SELECT * FROM users WHERE blood_type = %s AND hold_points >= %s'
cursor.execute(sql, ('A', 100)) # 參數順序對應 %s 的位置
```

### 4.3 查詢資料(SELECT)

執行 SELECT 後, 用 取單筆, 用 取多筆:

```
# 查詢單筆 (用 fetchone)
cursor.execute('SELECT * FROM users WHERE donor_id = %s', (donor_id,))
user = cursor.fetchone()
# user 是 dict (找到) 或 None (找不到)

if user is None:
    print('找不到此使用者')
else:
    print(user['name'])

# 查詢多筆 (用 fetchall)
cursor.execute(
    'SELECT * FROM donation_records WHERE donor_id = %s ORDER BY donation_date'
    'DESC',
    (donor_id,)
)
records = cursor.fetchall()
# records 是 list of dict (可能是空 list)

for record in records:
    print(record['donation_date'], record['category'])
```

### 4.4 新增資料(INSERT)

```
sql = '''
    INSERT INTO some_table (col_a, col_b, col_c)
    VALUES (%s, %s, %s)
'''
cursor.execute(sql, (value_a, value_b, value_c))

# INSERT / UPDATE / DELETE 之後都要 commit, 變更才會真正寫入資料庫
conn.commit()

# 取得剛插入的 AUTO_INCREMENT ID
new_id = cursor.lastrowid
```

### 4.5 更新資料(UPDATE)

```
# 固定欄位更新
cursor.execute(
    'UPDATE users SET hold_points = hold_points + %s WHERE donor_id = %s',
    (points_to_add, donor_id)
)
conn.commit()

# 動態欄位更新 (只更新有傳入的欄位)
# 參考第 2.3 節的技巧來組合 SET 子句
fields = data.model_dump(exclude_unset=True)
```

```
set_clause = ', '.join(f'{k} = %s' for k in fields)
values = list(fields.values()) + [record_id]

cursor.execute(f'UPDATE table_name SET {set_clause} WHERE id = %s', values)
conn.commit()
```

## 4.6 刪除資料(DELETE)

```
cursor.execute('DELETE FROM some_table WHERE id = %s', (record_id,))
conn.commit()

# cursor.rowcount 回傳受影響的列數
# 如果是 0, 表示沒有找到要刪除的資料
if cursor.rowcount == 0:
    print('找不到要刪除的資料')
```

## 4.7 Transaction: 把多個操作綁在一起

有時候一個 API 需要同時執行多個 SQL。例如新增捐血紀錄的同時，也要更新 USER 表的 Last\_Date。這兩個操作要同時成功或同時失敗，不能只做一半：

```
try:
    # 第一個操作
    cursor.execute('INSERT INTO donation_records ...', (...))

    # 第二個操作 (依賴第一個成功)
    cursor.execute('UPDATE users SET last_date = %s WHERE donor_id = %s', (...))

    conn.commit() # 兩個操作都成功才 commit
except Exception as e:
    conn.rollback() # 任一失敗就全部撤銷
    raise
```

### commit 與 rollback 的比喻

commit: 確認「這筆交易成立」，資料真正寫進資料庫。

rollback: 「取消這筆交易」，撤銷所有未 commit 的操作，回到操作前的狀態。

就像網路購物: 加入購物車(SQL 操作) → 確認結帳(commit);

如果付款失敗，訂單自動取消(rollback)。

## 4.8 共用連線函數

為了避免每支 API 都要重複寫連線的程式碼，把連線邏輯包成兩個函數放在 database.py，所有 router 都可以呼叫：

```
# database.py (架構示意，不是完整程式碼)

import mysql.connector
from dotenv import load_dotenv
import os
```

```
load_dotenv()

DB_CONFIG = {
    'host': os.getenv('DB_HOST'),
    # ... 其他設定 ...
}

def get_db():
    """建立並回傳 MySQL 連線與 cursor"""
    conn = mysql.connector.connect(**DB_CONFIG)
    cursor = conn.cursor(dictionary=True)
    return conn, cursor

def close_db(conn, cursor):
    """關閉連線，在 finally 裡呼叫"""
    cursor.close()
    conn.close()
```

在每支 API 裡的使用方式：

```
from database import get_db, close_db

@router.get('/something')
def some_api():
    conn, cursor = get_db()    # 開啟連線
    try:
        # 執行 SQL ...
        pass
    except HTTPException:
        raise
    except Exception as e:
        conn.rollback()
        raise HTTPException(status_code=500, detail=str(e))
    finally:
        close_db(conn, cursor) # 無論如何都要關閉
```

## 5. 專案架構說明

好的架構能讓多人協作不互相干擾，也讓程式碼更好維護。這個章節說明為什麼這樣切分檔案，以及每個檔案的職責。

### 5.1 目錄結構

```

blood_donation_api/
├── main.py           ← 程式進入點，把所有模組組合起來
├── database.py       ← 資料庫連線邏輯（共用）
├── schemas.py        ← 所有 Pydantic 資料格式（共用）
├── routers/          ← 每個模組一個檔案
│   ├── users.py      ← 模組 1a：使用者資料
│   ├── health.py     ← 模組 1b：健康履歷
│   ├── donations.py  ← 模組 2a：捐血紀錄
│   ├── sites.py      ← 模組 2b：捐血地點
│   └── rewards.py    ← 模組 3b：獎勵機制
├── .env              ← 資料庫帳密（不能上傳 Git!）
├── .gitignore        ← 告訴 Git 哪些檔案不要追蹤
└── requirements.txt  ← 套件清單
  
```

### 5.2 每個檔案的職責

| 檔案                        | 職責                                                                           |
|---------------------------|------------------------------------------------------------------------------|
| <code>main.py</code>      | 建立 FastAPI app, 把所有 router 掛載進來, 設定 API 基本資訊                                 |
| <code>database.py</code>  | 管理 MySQL 連線。提供 <code>get_db()</code> 和 <code>close_db()</code> 給所有 router 使用 |
| <code>schemas.py</code>   | 定義所有模組的 Pydantic Schema(Create、Update、Response)                              |
| <code>routers/*.py</code> | 每個檔案負責一個模組的 API, 只做「接收請求 → 執行 SQL → 回傳結果」                                    |
| <code>.env</code>         | 存放資料庫帳密等敏感設定, 絕對不能上傳到 GitHub                                                 |

### 5.3 一支 API 的完整資料流

以「新增捐血紀錄」為例，說明資料從前端到資料庫的完整流程：

```

前端送出  POST /donations
          Body: {donor_id: 1, donation_date: '2025-05-01', category: '全血'}
          |
          ↓
FastAPI 收到請求
          |
          1. 路由匹配：找到 @router.post('/') in donations.py
          2. Pydantic 驗證 Body 格式 (DonationRecordCreate)
          |
          ↓
donations.py 的 function 開始執行
          |
          3. get_db() 建立 MySQL 連線
          4. SELECT 確認 donor_id 存在
          5. INSERT 新增捐血紀錄
          6. UPDATE users SET last_date = ... (若本次日期較新)
          7. conn.commit()
          8. SELECT 查詢剛新增的資料
          9. close_db() 關閉連線
  
```

FastAPI 把查詢結果轉成 JSON

前端收到 201 Created  
{record\_id: 5, donor\_id: 1, donation\_date: '2025-05-01', ...}

## 5.4 main.py 的架構

main.py 只負責把所有 router 組合起來, 不寫任何業務邏輯:

```
# main.py 架構示意
from fastapi import FastAPI
from routers import users, health, donations, sites, rewards

app = FastAPI(title='捐血資料庫系統 API')

# 掛載各模組的 router, 並設定 URL 前綴
# prefix='/users' 代表 users.py 裡的所有路由都會加上 /users 前綴
app.include_router(users.router, prefix='/users', tags=['使用者管理'])
app.include_router(health.router, prefix='/health', tags=['健康履歷'])
app.include_router(donations.router, prefix='/donations', tags=['捐血紀錄'])
app.include_router(sites.router, prefix='/sites', tags=['捐血地點'])
app.include_router(rewards.router, prefix='/rewards', tags=['獎勵機制'])
```

## 6. 各模組開發指引

這個章節說明每個模組需要實作哪些 API、涉及哪些資料表、以及開發時需要注意的事項。程式碼由你自己根據前幾章的語法概念來撰寫。

### 6.1 模組 1a: 使用者基本資料管理

檔案: `routers/users.py` 資料表: **USER**

| API                                   | 說明與開發重點                                                                  |
|---------------------------------------|--------------------------------------------------------------------------|
| POST <code>/users</code>              | 新增使用者。新增前先 SELECT 確認 Email 是否已存在, 若重複回傳 400 錯誤                           |
| GET <code>/users/{donor_id}</code>    | 查詢單一使用者。若找不到(fetchone 回傳 None)回傳 404 錯誤                                  |
| PUT <code>/users/{donor_id}</code>    | 更新使用者。用 <code>model_dump(exclude_unset=True)</code> 取有傳入的欄位, 動態組合 SET 子句 |
| DELETE <code>/users/{donor_id}</code> | 刪除使用者。刪除前先確認存在, 刪除成功回傳 204 (不回傳任何內容)                                     |

USER 表涉及的欄位:

- 寫入時可設定: Name, Gender, Birthday, Blood\_Type, Phone, Email, Weight, Location, Drugs\_Record
- 系統自動管理: Donor\_ID (AUTO\_INCREMENT)、Hold\_Points (預設 0)、Last\_Date (捐血時更新)

### 6.2 模組 1b: 健康履歷管理

檔案: `routers/health.py` 資料表: **USER** (Drugs\_Record 欄位)

健康資料沒有獨立的表格, 直接儲存在 USER 表的 Drugs\_Record 欄位 (字串格式)。

| API                                    | 說明與開發重點                                            |
|----------------------------------------|----------------------------------------------------|
| POST <code>/health/{donor_id}</code>   | 新增健康紀錄, 實際上是 UPDATE USER 的 Drugs_Record 欄位 (附加或覆蓋) |
| GET <code>/health/{donor_id}</code>    | 讀取使用者的 Drugs_Record 欄位內容                           |
| PUT <code>/health/{donor_id}</code>    | 更新 Drugs_Record 欄位 (修正或補充紀錄)                       |
| DELETE <code>/health/{donor_id}</code> | 清空 Drugs_Record 欄位 (UPDATE 設為 NULL 或空字串)           |



 設計討論

Drugs\_Record 是字串欄位, 要和組員討論儲存格式, 例如用 JSON 字串或換行分隔。  
 例如: '{"medications": ["阿斯匹靈"], "travel": ["東南亞"]}'  
 格式統一之後, 所有 API 的讀寫邏輯才能一致。

## 6.3 模組 2a: 捐血紀錄管理

檔案: `routers/donations.py` 資料表: `DONATION_RECORD`、`USER`

| API                                         | 說明與開發重點                                                                                              |
|---------------------------------------------|------------------------------------------------------------------------------------------------------|
| POST <code>/donations</code>                | 新增捐血紀錄。同時要 UPDATE <code>USER</code> 的 <code>Last_Date</code> (若本次日期比原紀錄更新)。這兩個操作要包在同一個 transaction 裡 |
| GET <code>/donations/user/{donor_id}</code> | 查詢某使用者的所有捐血紀錄, 建議加 <code>ORDER BY donation_date DESC</code> 讓最新的排前面                                  |
| PUT <code>/donations/{record_id}</code>     | 更新捐血紀錄, 一樣用動態欄位更新                                                                                    |
| DELETE <code>/donations/{record_id}</code>  | 刪除捐血紀錄                                                                                               |

`DONATION_RECORD` 表涉及的欄位: `Record_ID`、`Donation_Date`、`Address`、`Category`

注意: 新增時需同時更新 `USER.Last_Date`, 這是兩張表的操作, 要用 transaction 確保一致性 (參考第 4.7 節)。

## 6.4 模組 2b: 捐血地點查詢

檔案: `routers/sites.py` 資料表: `DONATION_SITE`、`TRANSPORTATION`

| API                               | 說明與開發重點                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------|
| GET <code>/sites</code>           | 列出所有捐血地點。可加 <code>category</code> 查詢參數篩選 (捐血站 or 捐血車), SQL 用 <code>WHERE category = %s</code> |
| GET <code>/sites/{site_id}</code> | 查詢單一地點, 同時 JOIN <code>TRANSPORTATION</code> 取得交通資訊                                            |
| GET <code>/sites/nearby</code>    | 附近地點查詢。接收使用者的緯度/經度, 在 SQL 裡用 Haversine 公式計算距離, 只回傳範圍內的地點                                      |

附近查詢的 Haversine SQL 公式(在 MySQL 裡直接計算距離, 效能比撈回 Python 再算好很多):

```
-- 6371 是地球半徑 (公里)
-- %s 依序帶入: 使用者緯度、使用者經度、使用者緯度、搜尋半徑
SELECT *,
    (6371 * ACOS(
        COS(RADIANS(%s)) * COS(RADIANS(Latitude))
        * COS(RADIANS(Longitude) - RADIANS(%s))
        + SIN(RADIANS(%s)) * SIN(RADIANS(Latitude))
    )) AS distance_km
FROM DONATION_SITE
WHERE Latitude IS NOT NULL
HAVING distance_km <= %s
ORDER BY distance_km ASC
```

JOIN TRANSPORTATION 的寫法(取得地點的交通方式):

```
SELECT s.*, t.Trans_Type
FROM DONATION_SITE s
LEFT JOIN TRANSPORTATION t ON s.Site_ID = t.Location_ID
WHERE s.Site_ID = %s
```

## 6.5 模組 3b: 獎勵機制管理

檔案: `routers/rewards.py` 資料表: `USER(Hold_Points)`、`GIFT`

| API                                     | 說明與開發重點                                                                 |
|-----------------------------------------|-------------------------------------------------------------------------|
| GET<br><code>/rewards/{donor_id}</code> | 查詢使用者目前的 Hold_Points, 並在 Python 裡計算對應等級(用 if/elif 判斷)                   |
| POST <code>/rewards/add</code>          | 新增點數。用 UPDATE users SET hold_points = hold_points + %s 直接累加, 不需要先查出來再計算 |
| POST <code>/rewards/redeem</code>       | 兌換點數。先 SELECT 確認點數足夠, 不足回傳 400 錯誤; 足夠才 UPDATE 扣除                        |
| GET <code>/rewards/gifts</code>         | 查詢所有可兌換的贈品(SELECT FROM GIFT), 讓使用者知道要集多少點                               |

等級計算邏輯(在 Python 裡用 if/elif 實作, 依照 Hold\_Points 決定稱號):

```
# 等級定義由你們自己決定, 以下只是示意
def get_level(points):
    if points < 100:
        return '___'
    elif points < 300:
        return '___'
    # ... 繼續往下定義
```

 兌換點數要注意的順序

1. SELECT 查詢目前 Hold\_Points
  2. 在 Python 裡判斷點數是否足夠
  3. 足夠才執行 UPDATE 扣除
  4. 這三步也要包在 try/except/finally 裡
- 千萬不要先扣點再判斷，這樣點數可能會變成負數！

## 7. 測試你寫的 API

寫完 API 之後要測試，確認每個功能正確。最簡單的方式是用 FastAPI 內建的 Swagger UI。

### 7.1 Swagger UI

啟動伺服器後，FastAPI 會自動產生互動式文件，可以直接在瀏覽器裡測試所有 API：

```
# 啟動伺服器（在專案根目錄執行）
uvicorn main:app --reload

# 開啟瀏覽器前往
http://127.0.0.1:8000/docs
```

測試步驟：

- 點開要測試的 API
- 點右上角「Try it out」
- 填入測試資料，點「Execute」
- 檢查 Response Code 和 Response Body 是否符合預期

### 7.2 測試每個 API 時要驗證的事

| 測試情境       | 預期結果                          |
|------------|-------------------------------|
| 正常新增(資料完整) | 回傳 201, body 包含新資料(含自動產生的 ID) |
| 查詢存在的資料    | 回傳 200 和資料內容                  |
| 查詢不存在的 ID  | 回傳 404, detail 說明找不到          |
| 更新只傳部分欄位   | 只有傳入的欄位被更新, 其他欄位維持原值          |
| 刪除成功       | 回傳 204, 沒有 body               |
| Email 重複註冊 | 回傳 400, detail 說明 Email 已存在   |
| 兌換點數超過餘額   | 回傳 400, detail 說明點數不足         |
| 缺少必填欄位     | 回傳 422(FastAPI 自動驗證), 不需要自己處理 |

## 8. 環境設定步驟

### 8.1 安裝套件

```
# 啟動虛擬環境
python -m venv venv
source venv/bin/activate      # Mac/Linux
venv\Scripts\activate        # Windows

# 安裝所需套件
pip install fastapi uvicorn mysql-connector-python pydantic python-dotenv
```

### 8.2 建立 .gitignore

在專案根目錄建立 .gitignore, 確保帳密和暫存檔不被上傳:

```
venv/
.env
__pycache__/
*.pyc
.pytest_cache/
```

### 8.3 Git 協作流程

每個人在自己的 branch 上開發, 完成後再合併:

```
# 從 main 建立自己的 branch
git checkout -b feature/users-module

# 開發完成後提交
git add .
git commit -m 'feat: 完成使用者 CRUD'
git push origin feature/users-module

# 每次開始工作前先同步最新程式碼
git pull origin main
```



#### 共用檔案的協作規則

database.py 和 schemas.py 是共用檔案, 修改前先在群組裡說一聲。  
每個人只負責修改自己的 router 檔案

## 9. 開發前確認清單

### ✓ 每次開始開發前確認

- ☐ 虛擬環境已啟動(命令列前有 (venv))
- ☐ .env 檔案存在且填好資料庫帳密
- ☐ MySQL 服務已啟動, blood\_donation\_db 資料庫已建立
- ☐ git pull 取得最新程式碼
- ☐ uvicorn main:app --reload 可以正常啟動
- ☐ <http://127.0.0.1:8000/docs> 能正常顯示